

1. Create a Pandas Series containing the marks: 78, 85, 92, 67, 88

Display:

Complete Series
Values
Index
Data type
Mean
Maximum
Minimum

```
import pandas as pd  
import numpy as np
```

```
a = [78, 85, 92, 67, 88]  
print(a)  
b = pd.Series(a)  
print("\nComplete Series :\n", b)  
print("\nValues :\n", b.values)  
print("\nIndex :\n", b.index)  
print("\nData type:\n", b.dtype)  
print("\nMean : \n", b.mean())  
print("\nMaximum :\n", b.max())  
print("\nMinimum :\n", b.min())
```

```
[78, 85, 92, 67, 88]
```

Complete Series :

```
0    78
```

```
1    85
```

```
2    92
```

```
3    67
```

```
4    88
```

```
dtype: int64
```

Values :

```
[78 85 92 67 88]
```

Index :

```
RangeIndex(start=0, stop=5, step=1)
```

Data type:

```
int64
```

Mean :

```
82.0
```

Maximum :
92

Minimum :
67

2. Create a Series containing:

Student	Marks
Asha	85
Ravi	78
Meena	92
Kiran	74
Arun	88

Display the marks of Meena.

```
student = {'Student' : 'Marks' , 'Asha' : 85, 'Ravi' : 78, 'Meena' : 92, 'Kiran' : 74, 'Arun' : 88}
table = pd.Series(student)
marks = table['Meena']
print("Table : \n",table)

print("\nMarks of Meena : ", marks)
```

```
Table :
Student    Marks
Asha       85
Ravi       78
Meena      92
Kiran      74
Arun       88
dtype: object
```

Marks of Meena : 92

3. Create the following DataFrame:

Name	Age	Department	Marks
Asha	22	CSE	85
Ravi	23	ECE	78
Meena	21	CSE	92
Kiran	24	ME	74
Arun	22	ECE	88

Display:

Complete DataFrame

First 3 rows

Last 2 rows

Shape

Column names

Data types

```
Student = {'Name' : ['Asha', 'Ravi', 'Meena', 'Kiran', 'Arun'], 'Age': [22, 23, 21, 24, 22], 'Department': ['CSE', 'ECE', 'CSE', 'ME', 'ECE'], 'Marks': [85, 78, 92, 74, 88]}
table = pd.DataFrame(Student)
print("Table : \n", table)
```

Table :

	Name	Age	Department	Marks
0	Asha	22	CSE	85
1	Ravi	23	ECE	78
2	Meena	21	CSE	92
3	Kiran	24	ME	74
4	Arun	22	ECE	88

```
print("First 3 rows :\n ", table.head(3) )
```

First 3 rows :

	Name	Age	Department	Marks
0	Asha	22	CSE	85
1	Ravi	23	ECE	78
2	Meena	21	CSE	92

```
print("Last 2 rows :\n ", table.tail(2))
```

Last 2 rows :

	Name	Age	Department	Marks
3	Kiran	24	ME	74
4	Arun	22	ECE	88

```
print("Shape : ", table.shape)
```

Shape : (5, 4)

```
print("Column names :\n ", table.columns)
```

Column names :

```
Index(['Name', 'Age', 'Department', 'Marks'], dtype='object')
```

```
print("Data types :\n ", table.dtypes)
```

Data types :

Name	object
Age	int64

```

Department    object
Marks         int64
dtype: object

```

4. Using the previous DataFrame, display:

```

info()
describe()
Number of rows
Number of columns

```

```
print("Info : \n", table.info())
```

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 5 entries, 0 to 4
Data columns (total 4 columns):
#   Column      Non-Null Count  Dtype
---  -
0   Name        5 non-null     object
1   Age         5 non-null     int64
2   Department  5 non-null     object
3   Marks       5 non-null     int64
dtypes: int64(2), object(2)
memory usage: 292.0+ bytes
Info :
None

```

```
print("Describe : ", table.describe())
```

```

Describe :           Age      Marks
count    5.000000    5.000000
mean     22.400000    83.400000
std       1.140175     7.334848
min       21.000000    74.000000
25%       22.000000    78.000000
50%       22.000000    85.000000
75%       23.000000    88.000000
max       24.000000    92.000000

```

```
print("Number of rows : ", table.shape[0])
```

```
Number of rows : 5
```

```
print("Number of columns : ", table.shape[1])
```

```
Number of columns : 4
```

Using df, display:

Name column
 Marks column
 Name and Marks together

```
df = table
print("Name :\n", df['Name'])
```

```
Name :
0    Asha
1    Ravi
2    Meena
3    Kiran
4    Arun
Name: Name, dtype: object
```

```
print("Marks :\n", df['Marks'])
```

```
Marks :
0    85
1    78
2    92
3    74
4    88
Name: Marks, dtype: int64
```

```
print("Name and Marks : \n", df[['Name', 'Marks']])
```

```
Name and Marks :
   Name  Marks
0  Asha    85
1  Ravi    78
2  Meena   92
3  Kiran    74
4  Arun    88
```

6. Display:

First row using iloc
 Third row using iloc
 Rows 1 to 3
 Name and Marks columns using loc

```
print("First row :\n", df.iloc[0])
```

```
First row :
Name      Asha
```

```
Age      22
Department CSE
Marks    85
Name: 0, dtype: object
```

```
print("Third row :\n", df.iloc[2])
```

```
Third row :
Name      Meena
Age      21
Department CSE
Marks    92
Name: 2, dtype: object
```

```
print("Rows from 1 to 3 :\n", df.iloc[0:3])
```

```
Rows from 1 to 3 :
   Name  Age Department  Marks
0  Asha   22         CSE     85
1   Ravi   23         ECE     78
2  Meena   21         CSE     92
```

```
print("Name and marks columns :\n", df[['Name', 'Marks']])
```

```
Name and marks columns :
   Name  Marks
0  Asha     85
1   Ravi     78
2  Meena     92
3  Kiran     74
4   Arun     88
```

7. Find:

```
Students scoring above 80.
Students scoring between 75 and 90.
Students belonging to CSE.
CSE students scoring above 80.
```

```
print("Students scoring above 80 :\n", df[df['Marks'] > 80])
```

```
Students scoring above 80 :
   Name  Age Department  Marks
0  Asha   22         CSE     85
2  Meena   21         CSE     92
4   Arun   22         ECE     88
```

```
print("Students scoring between 75 and 90 :\n", df[(df['Marks'] > 75) & (df['
```

```
Students scoring between 75 and 90 :
      Name  Age Department  Marks
0  Asha    22         CSE     85
1  Ravi    23         ECE     78
4  Arun    22         ECE     88
```

```
print("Students belonging to CSE :\n", df[df['Department'] == 'CSE'])
```

```
Students belonging to CSE :
      Name  Age Department  Marks
0  Asha    22         CSE     85
2  Meena   21         CSE     92
```

```
print("CSE students scoring above 80 :\n ",df[(df['Department'] == 'CSE') & (df
```

```
CSE students scoring above 80 :
      Name  Age Department  Marks
0  Asha    22         CSE     85
2  Meena   21         CSE     92
```

8. Display students belonging to either CSE or ECE.

```
print("Students belonging to either CSE or ECE :\n ", df[(df['Department'] == 'CSE') | (df['Department'] == 'ECE')])
```

```
Students belonging to either CSE or ECE :
      Name  Age Department  Marks
0  Asha    22         CSE     85
1  Ravi    23         ECE     78
2  Meena   21         CSE     92
4  Arun    22         ECE     88
```

9. Add a new column called Result. Students scoring 75 or above should receive "Pass", otherwise "Fail".

```
df['Result'] = np.where(df['Marks'] >= 75, 'Pass', 'Fail')
print(df)
```

```
      Name  Age Department  Marks  Result  Grade
0  Asha    22         CSE     85    Pass     B
1  Ravi    23         ECE     78    Pass     C
2  Meena   21         CSE     92    Pass     A
3  Kiran   24         ME     74    Fail     C
4  Arun    22         ECE     88    Pass     B
```

10. Assign grades according to:

Marks	Grade
≥ 90	A

Marks Grade

≥ 80 B

≥ 70 C

< 70 D

```
def grade(marks):
    if marks > 90:
        return 'A'
    elif marks >= 80:
        return 'B'
    elif marks >= 70:
        return 'C'
    else:
        return 'D'
```

```
df['Grade'] = df['Marks'].apply(grade)
print(df)
```

	Name	Age	Department	Marks	Result	Grade
0	Asha	22	CSE	85	Pass	B
1	Ravi	23	ECE	78	Pass	C
2	Meena	21	CSE	92	Pass	A
3	Kiran	24	ME	74	Fail	C
4	Arun	22	ECE	88	Pass	B

11. Sort students:

By marks in ascending order.
 By marks in descending order.
 By Department and then Marks.

```
print("Sorting students by marks in ascending order :\n ", df.sort_values(by =
```

```
Sorting students by marks in ascending order :
      Name  Age  Department  Marks  Result  Grade
3  Kiran   24         ME     74   Fail     C
1   Ravi   23         ECE     78   Pass     C
0   Asha   22         CSE     85   Pass     B
4   Arun   22         ECE     88   Pass     B
2  Meena   21         CSE     92   Pass     A
```

```
print("Sorting students by marks in descending order :\n ", df.sort_values(by =
```

```
Sorting students by marks in descending order :
      Name  Age  Department  Marks  Result  Grade
2  Meena   21         CSE     92   Pass     A
4   Arun   22         ECE     88   Pass     B
```

0	Asha	22	CSE	85	Pass	B
1	Ravi	23	ECE	78	Pass	C
3	Kiran	24	ME	74	Fail	C

```
print("Sorting students by Department and then Marks :\n", df.sort_values(by =
```

Sorting students by Department and then Marks :

	Name	Age	Department	Marks	Result	Grade
0	Asha	22	CSE	85	Pass	B
2	Meena	21	CSE	92	Pass	A
1	Ravi	23	ECE	78	Pass	C
4	Arun	22	ECE	88	Pass	B
3	Kiran	24	ME	74	Fail	C

12. Consider:

```
data = {
```

```
    "Name": ["Asha", "Ravi", "Meena", "Kiran", "Arun"],
    "Age": [22, 23, np.nan, 24, 22],
    "Marks": [85, np.nan, 92, 74, 88]
}
```

```
students = pd.DataFrame(data)
```

Find:

Missing values

Number of missing values in each column

Rows containing missing values

```
data = {

    "Name": ["Asha", "Ravi", "Meena", "Kiran", "Arun"],
    "Age": [22, 23, np.nan, 24, 22],
    "Marks": [85, np.nan, 92, 74, 88]

}
```

```
students = pd.DataFrame(data)
print(students)
```

	Name	Age	Marks
0	Asha	22.0	85.0
1	Ravi	23.0	NaN
2	Meena	NaN	92.0

```
3 Kiran 24.0 74.0
4 Arun 22.0 88.0
```

```
print("Missing values :\n ", students.isnull())
```

```
Missing values :
      Name    Age  Marks
0  False  False  False
1  False  False   True
2  False   True  False
3  False  False  False
4  False  False  False
```

```
print("Number of missing values in each column :\n", students.isnull().sum())
```

```
Number of missing values in each column :
      Name    0
      Age     1
      Marks   1
dtype: int64
```

```
print("Rows containing missing values :\n",students[students.isnull().any(axis
```

```
Rows containing missing values :
      Name    Age  Marks
1   Ravi  23.0   NaN
2  Meena   NaN  92.0
```

13. Replace:

```
Missing Age with average Age
Missing Marks with average Marks
```

```
students_replaced_values = students.copy()

average_age = students_replaced_values['Age'].mean()
average_marks = students_replaced_values['Marks'].mean()

students_replaced_values['Age'] = students_replaced_values['Age'].fillna(average_age)
students_replaced_values['Marks'] = students_replaced_values['Marks'].fillna(average_marks)

print(students_replaced_values)
```

```
      Name    Age  Marks
0   Asha  22.00  85.00
1   Ravi  23.00  84.75
2  Meena  22.75  92.00
3  Kiran  24.00  74.00
4   Arun  22.00  88.00
```

14. Remove rows containing missing values.

```
students_cleaned = students.dropna()

print("Table before removing missing values :\n", students)
print("\nTable after removing missing values \n: ", students_cleaned)
```

Table before removing missing values :

	Name	Age	Marks
0	Asha	22.0	85.0
1	Ravi	23.0	NaN
2	Meena	NaN	92.0
3	Kiran	24.0	74.0
4	Arun	22.0	88.0

Table after removing missing values

	Name	Age	Marks
0	Asha	22.0	85.0
3	Kiran	24.0	74.0
4	Arun	22.0	88.0

15. Consider:

```
data = {

    "Name": ["Asha", "Ravi", "Meena", "Asha", "Arun"],
    "Marks": [85, 78, 92, 85, 88]
}
```

```
df2 = pd.DataFrame(data)
```

Find duplicates:

Display duplicate rows:

Remove duplicates:

```
data = {

    "Name": ["Asha", "Ravi", "Meena", "Asha", "Arun"],
    "Marks": [85, 78, 92, 85, 88]
}
```

```
df2 = pd.DataFrame(data)
print(df2)
```

	Name	Marks
0	Asha	85

```

1  Ravi    78
2  Meena   92
3  Asha    85
4  Arun    88

```

```
print("Duplicates :\n", df2.duplicated())
```

```

Duplicates :
0    False
1    False
2    False
3     True
4    False
dtype: bool

```

```
print("Duplicate rows :\n", df2[df2.duplicated()])
```

```

Duplicate rows :
   Name  Marks
3  Asha    85

```

```
df2_cleaned = df2.copy()
```

```

df2_cleaned = df2_cleaned.drop_duplicates()
print("Tbale before removing duplicates :\n", df2)
print("\nTable after removing duplicates :\n", df2_cleaned)

```

```

Tbale before removing duplicates :
   Name  Marks
0  Asha    85
1  Ravi    78
2  Meena   92
3  Asha    85
4  Arun    88

```

```

Table after removing duplicates :
   Name  Marks
0  Asha    85
1  Ravi    78
2  Meena   92
4  Arun    88

```

16. Given:

```

marks = pd.DataFrame({
    "Student": ["A", "B", "C", "D", "E"],
    "Marks": [78, 85, 92, 67, 88]
})

```

Calculate:

Mean
Median
Maximum
Minimum
Standard deviation
Sum

```
marks = pd.DataFrame({
    "Student": ["A", "B", "C", "D", "E"],
    "Marks": [78, 85, 92, 67, 88]
})

print("Mean : ", marks['Marks'].mean())
print("Median : ", marks['Marks'].median())
print("Maximum : ", marks['Marks'].max())
print("Minimum : ", marks['Marks'].min())
print("Standard deviation : ", marks['Marks'].std())
print("Sum : ", marks['Marks'].sum())
```

```
Mean : 82.0
Median : 85.0
Maximum : 92
Minimum : 67
Standard deviation : 9.82344135219425
Sum : 410
```

17. Consider:

```
data = {
    "Name": [
        "Asha", "Ravi", "Meena",
        "Kiran", "Arun", "Divya"
    ],
    "Department": [
        "CSE", "ECE", "CSE",
        "ME", "ECE", "CSE"
    ],
    "Marks": [
        85, 78, 92,
        74, 88, 80
    ]
}
```

```
}
```

```
df = pd.DataFrame(data)
```

Find:

Average marks by department

Maximum marks by department

Number of students in each department

For each department, calculate:

Mean

Minimum

Maximum

Count

```
data = {  
    "Name": [  
        "Asha", "Ravi", "Meena",  
        "Kiran", "Arun", "Divya"  
    ],  
    "Department": [  
        "CSE", "ECE", "CSE",  
        "ME", "ECE", "CSE"  
    ],  
    "Marks": [  
        85, 78, 92,  
        74, 88, 80  
    ]  
}  
  
df = pd.DataFrame(data)  
df
```

	Name	Department	Marks	
0	Asha	CSE	85	
1	Ravi	ECE	78	
2	Meena	CSE	92	
3	Kiran	ME	74	
4	Arun	ECE	88	
5	Divya	CSE	80	

Next steps:

[Generate code with df](#)[New interactive sheet](#)

```
print("Average marks by department :\n", df.groupby('Department')['Marks'].mean())
```

```
Average marks by department :
Department
CSE      85.666667
ECE      83.000000
ME       74.000000
Name: Marks, dtype: float64
```

```
print("Maximum marks by department :\n", df.groupby('Department')['Marks'].max())
```

```
Maximum marks by department :
Department
CSE      92
ECE      88
ME       74
Name: Marks, dtype: int64
```

```
print("Number of students in each department :\n", df.groupby('Department')['Name'].count())
```

```
Number of students in each department :
Department
CSE      3
ECE      2
ME       1
Name: Name, dtype: int64
```

```
print("For each department : ")
print("Mean :\n", df.groupby('Department')['Marks'].mean())
print("\n\nMinimum :\n", df.groupby('Department')['Marks'].min())
print("\n\nCount :\n", df.groupby('Department')['Marks'].count())
```

```
For each department :
Mean :
Department
CSE      85.666667
```

```
ECE    83.000000
ME     74.000000
Name: Marks, dtype: float64
```

```
Minimum :
  Department
CSE      92
ECE      88
ME       74
Name: Marks, dtype: int64
```

```
Count :
  Department
CSE       3
ECE       2
ME        1
Name: Marks, dtype: int64
```

18. Create:

```
students = pd.DataFrame({
    "StudentID": [101, 102, 103, 104],
    "Name": ["Asha", "Ravi", "Meena", "Kiran"]
})

marks = pd.DataFrame({
    "StudentID": [101, 102, 103, 104],
    "Marks": [85, 78, 92, 74]
})
```

Combine both DataFrames using StudentID.

```
students = pd.DataFrame({
    "StudentID": [101, 102, 103, 104],
    "Name": ["Asha", "Ravi", "Meena", "Kiran"]
})

marks = pd.DataFrame({
    "StudentID": [101, 102, 103, 104],
    "Marks": [85, 78, 92, 74]
})
```

```
combined_df = pd.merge(students, marks, on = 'StudentID')
print("Combined :\n", combined_df)
```

```
Combined :
  StudentID  Name  Marks
```

0	101	Asha	85
1	102	Ravi	78
2	103	Meena	92
3	104	Kiran	74

19. Use the following dataset:

```
data = {  
    "Name": [  
        "Asha", "Ravi", "Meena",  
        "Kiran", "Arun", "Divya",  
        "John", "Priya"  
    ],  
    "Department": [  
        "CSE", "ECE", "CSE",  
        "ME", "ECE", "CSE",  
        "ME", "ECE"  
    ],  
    "Attendance": [  
        92, 75, 96, 68,  
        82, 88, 72, 95  
    ],  
    "Internal": [  
        85, 72, 94, 65,  
        80, 86, 70, 91  
    ],  
    "FinalMarks": [  
        88, 74, 96, 62,  
        83, 89, 68, 93  
    ]  
}
```

```
df = pd.DataFrame(data)
```

Perform the following analysis:

Display the first five records.

Find the dimensions of the dataset.

Find the average final mark.

Find the highest-scoring student.

Find the lowest-scoring student.
Display students scoring above 80.
Find average marks by department.
Find students having attendance above 80%.
Sort students by FinalMarks.
Add a Result column: Pass if FinalMarks ≥ 70 , otherwise Fail.
Add a performance category:

 $\geq 90 \rightarrow$ Excellent
 $\geq 80 \rightarrow$ Very Good
 $\geq 70 \rightarrow$ Good
 $< 70 \rightarrow$ Needs Improvement

Find the department having the highest average FinalMarks.

```
data = {
    "Name": [
        "Asha", "Ravi", "Meena",
        "Kiran", "Arun", "Divya",
        "John", "Priya"
    ],

    "Department": [
        "CSE", "ECE", "CSE",
        "ME", "ECE", "CSE",
        "ME", "ECE"
    ],

    "Attendance": [
        92, 75, 96, 68,
        82, 88, 72, 95
    ],

    "Internal": [
        85, 72, 94, 65,
        80, 86, 70, 91
    ],

    "FinalMarks": [
        88, 74, 96, 62,
        83, 89, 68, 93
    ]
}

df = pd.DataFrame(data)
df
```

	Name	Department	Attendance	Internal	FinalMarks	
0	Asha	CSE	92	85	88	
1	Ravi	ECE	75	72	74	
2	Meena	CSE	96	94	96	
3	Kiran	ME	68	65	62	
4	Arun	ECE	82	80	83	
5	Divya	CSE	88	86	89	
6	John	ME	72	70	68	
7	Priya	ECE	95	91	93	

Next steps:

[Generate code with df](#)[New interactive sheet](#)

```
print("First five records :\n", df.head(5))
```

First five records :

	Name	Department	Attendance	Internal	FinalMarks
0	Asha	CSE	92	85	88
1	Ravi	ECE	75	72	74
2	Meena	CSE	96	94	96
3	Kiran	ME	68	65	62
4	Arun	ECE	82	80	83

```
print("Dimensions of the dataset : ", df.shape)
```

Dimensions of the dataset : (8, 5)

```
print("Average final mark : ", df['FinalMarks'].mean())
```

Average final mark : 81.625

```
print("Highest-scoring student :\n ", df[df['FinalMarks'] == df['FinalMarks'].n
```

Highest-scoring student :

	Name	Department	Attendance	Internal	FinalMarks
2	Meena	CSE	96	94	96

```
print("Lowest-scoring student :\n ", df[df['FinalMarks'] == df['FinalMarks'].mi
```

Lowest-scoring student :

	Name	Department	Attendance	Internal	FinalMarks
3	Kiran	ME	68	65	62

```
print("Students scoring above 80 :\n ", df[df['FinalMarks'] > 80])
```

Students scoring above 80 :

	Name	Department	Attendance	Internal	FinalMarks
0	Asha	CSE	92	85	88
2	Meena	CSE	96	94	96
4	Arun	ECE	82	80	83
5	Divya	CSE	88	86	89
7	Priya	ECE	95	91	93

```
print("Average marks by department :\n ", df.groupby('Department')['FinalMarks'
```

Average marks by department :

Department	
CSE	91.000000
ECE	83.333333
ME	65.000000

Name: FinalMarks, dtype: float64

```
print("Students having attendance above 80% :\n ", df[df['Attendance'] > 80])
```

Students having attendance above 80% :

	Name	Department	Attendance	Internal	FinalMarks
0	Asha	CSE	92	85	88
2	Meena	CSE	96	94	96
4	Arun	ECE	82	80	83
5	Divya	CSE	88	86	89
7	Priya	ECE	95	91	93

```
print("Students sorted by FinalMarks :\n",df.sort_values(by = 'FinalMarks'))
```

Students sorted by FinalMarks :

	Name	Department	Attendance	Internal	FinalMarks
3	Kiran	ME	68	65	62
6	John	ME	72	70	68
1	Ravi	ECE	75	72	74
4	Arun	ECE	82	80	83
0	Asha	CSE	92	85	88
5	Divya	CSE	88	86	89
7	Priya	ECE	95	91	93
2	Meena	CSE	96	94	96

```
df['Result'] = np.where(df['FinalMarks'] >= 70, 'Pass', 'Fail')
df
```

	Name	Department	Attendance	Internal	FinalMarks	Result	
0	Asha	CSE	92	85	88	Pass	
1	Ravi	ECE	75	72	74	Pass	
2	Meena	CSE	96	94	96	Pass	
3	Kiran	ME	68	65	62	Fail	
4	Arun	ECE	82	80	83	Pass	
5	Divya	CSE	88	86	89	Pass	
6	John	ME	72	70	68	Fail	
7	Priya	ECE	95	91	93	Pass	

Next steps:

[Generate code with df](#)

[New interactive sheet](#)

```
conditions = [  
    (df['FinalMarks'] >= 90),  
    (df['FinalMarks'] >= 80),  
    (df['FinalMarks'] >= 70),  
    (df['FinalMarks'] < 70)  
]  
  
choices = ['Excellent', 'Very Good', 'Good', 'Needs Improvement']  
  
df['Performance'] = np.select(conditions, choices, default = 'Needs Improvement'  
  
df
```

	Name	Department	Attendance	Internal	FinalMarks	Result	Performance	
0	Asha	CSE	92	85	88	Pass	Very Good	
1	Ravi	ECE	75	72	74	Pass	Good	
2	Meena	CSE	96	94	96	Pass	Excellent	
3	Kiran	ME	68	65	62	Fail	Needs Improvement	
4	Arun	ECE	82	80	83	Pass	Very Good	
5	Divya	CSE	88	86	89	Pass	Very Good	
6	John	ME	72	70	68	Fail	Needs Improvement	

Next steps:

[Generate code with df](#)

[New interactive sheet](#)

```
department_average = df.groupby('Department')['FinalMarks'].mean()
print("Department having the highest FinalMarks :\n", department_average.loc[[c
```

```
Department having the highest FinalMarks :
Department
CSE      91.0
Name: FinalMarks, dtype: float64
```

20. Create:

```
employees = pd.DataFrame({
    "Employee": [
        "E01", "E02", "E03", "E04",
        "E05", "E06", "E07", "E08"
    ],
    "Department": [
        "IT", "HR", "IT", "Finance",
        "HR", "Finance", "IT", "Finance"
    ],
    "Experience": [
        5, 3, 7, 4,
        6, 8, 2, 5
    ],
    "Salary": [
        60000, 45000, 75000, 65000,
        55000, 85000, 48000, 70000
    ]
})
```

Determine:

Average salary.
 Maximum and minimum salary.
 Employee receiving the highest salary.
 Employees earning above ₹60,000.
 Average salary for each department.
 Average experience by department.
 Number of employees in each department.
 Sort employees by salary.
 Find IT employees with more than 3 years of experience.

Add a SalaryLevel column:

≥ ₹75,000 → High

₹50,000–₹74,999 → Medium

< ₹50,000 → Low

```
employees = pd.DataFrame({
    "Employee": [
        "E01", "E02", "E03", "E04",
        "E05", "E06", "E07", "E08"
    ],
    "Department": [
        "IT", "HR", "IT", "Finance",
        "HR", "Finance", "IT", "Finance"
    ],
    "Experience": [
        5, 3, 7, 4,
        6, 8, 2, 5
    ],
    "Salary": [
        60000, 45000, 75000, 65000,
        55000, 85000, 48000, 70000
    ]
})
employees
```

	Employee	Department	Experience	Salary	
0	E01	IT	5	60000	
1	E02	HR	3	45000	
2	E03	IT	7	75000	
3	E04	Finance	4	65000	
4	E05	HR	6	55000	
5	E06	Finance	8	85000	
6	E07	IT	2	48000	
7	E08	Finance	5	70000	

Next steps:

Generate code with employees

New interactive sheet

```
print("Average salary : ", employees['Salary'].mean())
```

```
Average salary : 62875.0
```

```
print("Maximum salary : ", employees['Salary'].max())
```

```
Maximum salary : 85000
```

```
print("Minimum salary : ", employees['Salary'].min())
```

```
Minimum salary : 45000
```

```
print("Employee receiving highest salary :\n ", employees[employees['Salary'] =
```

```
Employee receiving highest salary :
   Employee Department  Experience  Salary
5         E06      Finance           8   85000
```

```
print("Employee receiving highest salary :\n ", employees[employees['Salary'] >
```

```
Employee receiving highest salary :
   Employee Department  Experience  Salary
2         E03         IT           7   75000
3         E04      Finance           4   65000
5         E06      Finance           8   85000
7         E08      Finance           5   70000
```

```
print("Average salary for each department :\n ", employees.groupby('Department'
```

```
Average salary for each department :
   Department
Finance    73333.333333
HR         50000.000000
IT         61000.000000
Name: Salary, dtype: float64
```

```
print("Average experience by department :\n", employees.groupby('Department')['
```

```
Average experience by department :
   Department
Finance     5.7
HR          4.5
IT          4.7
Name: Experience, dtype: float64
```

```
print("Number of employees in each department :\n", employees.groupby('Departme
```

```
Number of employees in each department :
   Department
Finance     3
```

```
HR      2
IT      3
Name: Employee, dtype: int64
```

```
print("Employees sorted by salary :\n", employees.sort_values(by = 'Salary'))
```

```
Employees sorted by salary :
   Employee Department  Experience  Salary
1      E02          HR           3   45000
6      E07          IT           2   48000
4      E05          HR           6   55000
0      E01          IT           5   60000
3      E04  Finance           4   65000
7      E08  Finance           5   70000
2      E03          IT           7   75000
5      E06  Finance           8   85000
```

```
print("IT employees with more than 3 years of Experience :\n", employees[(employees['Experience'] > 3) & (employees['Department'] == 'IT')])
```

```
IT employees with more than 3 years of Experience :
   Employee Department  Experience  Salary
0      E01          IT           5   60000
2      E03          IT           7   75000
```

```
conditions = [
    (employees['Salary'] >= 75000),
    (employees['Salary'] >= 50000) & (employees['Salary'] < 75000),
]

choices = ['High', 'Medium']

employees['SalaryLevel'] = np.select(conditions, choices, default = 'Low')

print("Updated tabel with Salarylevel :\n", employees)
```

```
Updated tabel with Salarylevel :
   Employee Department  Experience  Salary SalaryLevel
0      E01          IT           5   60000      Medium
```